Students:
Nguyen Huu Truong Giang [u6110634@udg.edu]

Lab 4: Task-Priority kinematic control (1B)

1 Introduction

In controlling a robot arm, the robot sometimes has more degrees of freedom than required to complete a task. Therefore, we can utilize this redundancy to accomplish additional subtasks by using the Jacobian null-space projector. Moreover, various types of tasks can be defined, such as joint position and end-effector orientation. For each task, the Jacobian null-space projector is different, so it must be defined correctly.

In this lab, we introduce four types of tasks: end-effector position, end-effector orientation, end-effector configuration, and joint position. We also extend these tasks so that they can be applied not only to the end-effector but also to any link of the robot arm. In addition, we include a feedforward velocity vector for tracking objects and a gain matrix (K) to control how fast or slow each joint moves toward the desired position.

Section 2 presents the formulation of the error and the Jacobian null-space projector for each specific task. Section 3 illustrates the results under different scenarios and discusses the key observations from the experiment. Finally, Section 4 concludes the lab.

2 Methodology

The implementation was carried out in Python using the `NumPy` library for numerical computations and `Matplotlib` for visualizing the robot motion and results. The task definitions for end-effector position, end-effector orientation, end-effector configuration, and joint position tasks are described below.

2.1 End-effector position task

For the end-effector position task, we formulate the equations as follows:

$$\sigma_p = \sigma_p(\mathbf{q}) \in \mathbb{R}^{3\times 1}$$

$$\sigma_p = \eta_1 = \begin{bmatrix} x \\ y \\ z \end{bmatrix}, \qquad \tilde{\sigma}_p = \eta_{1,d} - \eta_1$$

$$J_p = J_p(\mathbf{q}) = J_v(\mathbf{q}) \in \mathbb{R}^{3\times n}$$

Where:

- $\tilde{\sigma}_p$: The error between the desired and current end-effector position.
- J_p : The linear velocity part of the end-effector Jacobian (top three rows).

Appendix A.1 shows the Python implementation of this task.

2.2 End-effector orientation task

To avoid gimbal lock in the orientation task, we transform the rotation matrix to a quaternion to calculate the error. The transformation is described as follows:

$$R = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}, \qquad w = \frac{1}{2}\sqrt{r_{11} + r_{22} + r_{33} + 1}$$

$$\epsilon = \frac{1}{2}\begin{bmatrix} \text{sgn}(r_{32} - r_{23})\sqrt{r_{11} - r_{22} - r_{33} + 1} \\ \text{sgn}(r_{13} - r_{31})\sqrt{r_{22} - r_{33} - r_{11} + 1} \\ \text{sgn}(r_{21} - r_{12})\sqrt{r_{33} - r_{11} - r_{22} + 1} \end{bmatrix}, \qquad \text{sgn}(x) = \begin{cases} 1, & x \geq 0 \\ -1, & x < 0 \end{cases}$$

Where:

- R : The rotation matrix.
- w : The scalar (real) part of the quaternion.
- ϵ : The vector (imaginary) part of the quaternion.

After obtaining the quaternion, the end-effector orientation task is defined as:

$$\sigma_o = \sigma_o(\mathbf{q}) \in \mathbb{R}^{3\times 1}$$

$$\tilde{\sigma}_o = w\epsilon_d - w_d\epsilon - \epsilon \times \epsilon_d, \qquad J_o = J_o(\mathbf{q}) = J_\omega(\mathbf{q}) \in \mathbb{R}^{3\times n}$$

Where:

- σ_o : The orientation error.
- J_o : The angular velocity part of the end-effector Jacobian (bottom three rows).

The Python implementation of this task is shown in Appendix A.2.

2.3 End-effector configuration task

This task combines the end-effector position and orientation tasks. The formulation is as follows:

$$\sigma_c = \sigma_c(\mathbf{q}) \in \mathbb{R}^{6\times 1}$$

$$\tilde{\sigma}_c = \begin{bmatrix} \eta_{1,d} - \eta_1 \\ w\epsilon_d - w_d\epsilon - \epsilon \times \epsilon_d \end{bmatrix}, \qquad J_c = J_c(\mathbf{q}) = J(\mathbf{q}) \in \mathbb{R}^{6\times n}$$

Where:

- $\tilde{\sigma}_c$: The combined error of end-effector position and orientation.
- J_c : The full end-effector Jacobian.

The Python implementation of this task is shown in Appendix A.3.

2.4 Joint position task

The joint position task is defined as follows:

$$\sigma_{ji} = \sigma_{ji}(\mathbf{q}) \in \mathbb{R}$$

$$\tilde{\sigma}_{ji} = \sigma_{ji,d} - \sigma_{ji}$$

$$\begin{array}{c} \quad\;\; 0 \;\; 1 \;\; \ldots \;\; i \;\; \ldots \;\; n \\ J_{ji} = [\,0,\, 0,\, \ldots,\, 1,\, \ldots,\, 0\,] \in \mathbb{R}^{1\times n} \end{array} \quad \leftarrow \text{Single-entry row matrix}$$

Where:

- J_{ji} : The Jacobian mapping directly to the joint of interest.
- $\tilde{\sigma}_{ji}$: The joint position error between the desired and current joint position.

The Python implementation of this task is shown in Appendix A.4.

3 Results

Two main experiments were conducted in this study. In the first experiment, we performed four tasks hierarchies and observed the behavior of each tasks hierarchies. In the second experiment, we implemented a task-priority scheme, assigning the end-effector position as the higher-priority task and a specific link orientation as the lower-priority task, using different gain matrices K.

In this laboratory experiment, a 3-DOF planar robot with three revolute joints is used. The robot model, including its coordinate frames and Denavit–Hartenberg (DH) parameters, is illustrated in Fig. 1.

The robot configuration is initialized according to the DH parameters listed in Table 1.

DOF	θ_i	d_i	a_i	α_i	**Home**
1	q_1	0	$a_1 = 0.5$	0	0.2
2	q_2	0	$a_2 = 0.5$	0	0.5
3	q_3	0	$a_3 = 0.5$	0	0.2

Table 1: Denavit–Hartenberg parameters of the 3-DOF robot arm

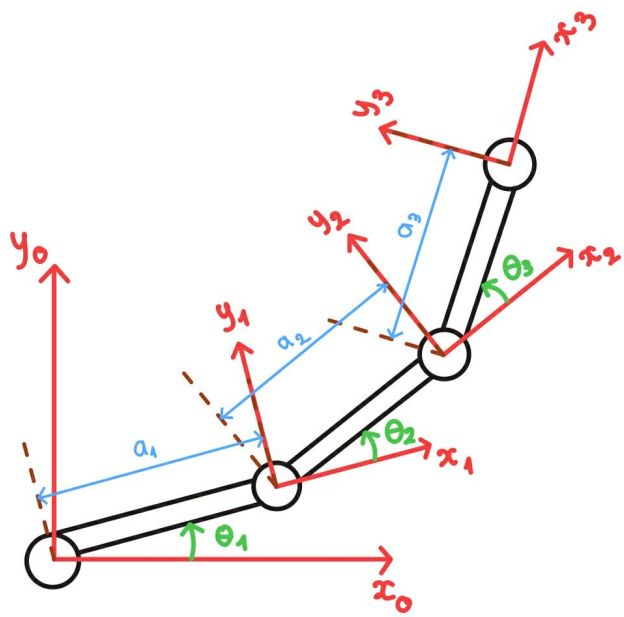


Figure 1: Visualization of the 3-DOF planar robot model

3.1 Four different task hierarchies

To conduct the first experiment, the total simulation time and the sampling interval are initialized. The parameters used in the simulation are summarized in Table 2.

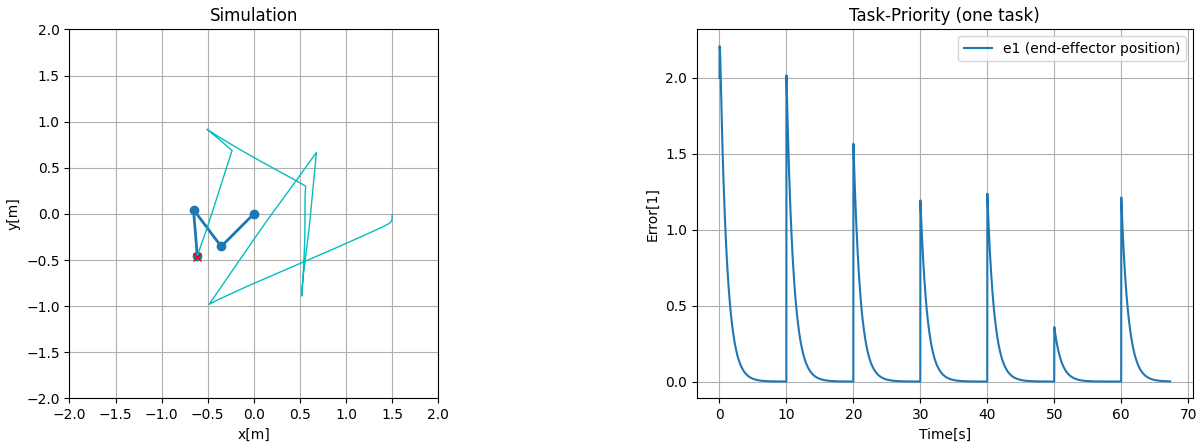
Desired end-effector orientation in end-effector orientation task	0
Desired end-effector orientation in configuration task	$\pi/2$
Desired joint 1 position	0
Sampling time	1/60
Total simulation time	10
Damping factor	0.1

Table 2: Parameters for the task hierarchy experiments

After setting these parameters, experiments are conducted using four different task hierarchies, as described below. The complete Python code implementation is provided in Appendices A.5.1 and A.5.2.

3.1.1 One task → 1: End-effector position

In this task hierarchy, only a single task is considered: moving the end-effector to a desired position. The visualization of the robot structure and its motion is shown in Fig. 2a. As shown in Fig. 2b, over a period of 10 seconds, the end-effector position error converges to zero, which confirms that the end-effector successfully reaches the desired position.



(a) Simulation of the manipulator.

(b) Evolution of the TP control errors.

Figure 2: End-effector position control task.

3.1.2 One task → 1: End-effector configuration

In this task hierarchy, the end-effector is required to reach both a desired position and a desired orientation simultaneously, since the task concerns the full end-effector configuration. The visualization of the robot structure and its motion is shown in Fig. 3a. As illustrated in Fig. 3b, both the position and orientation errors of the end-effector converge to zero when the desired configuration is achievable.

However, if either the position or orientation cannot be satisfied, the robot fails to complete the task, and neither error converges to zero. This behavior can be observed during the time intervals between 40 and 50, 50 and 60 seconds.

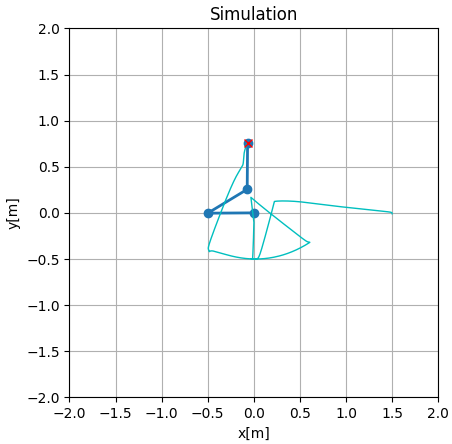
3.1.3 Two tasks → 1: End-effector position, 2: End-effector orientation

This task hierarchy differs from the previous one in that two tasks are defined with different priorities. The end-effector position task has higher priority than the end-effector orientation task. As a result, the robot first prioritizes reaching the desired end-effector position and only attempts to fulfill the orientation task if possible.

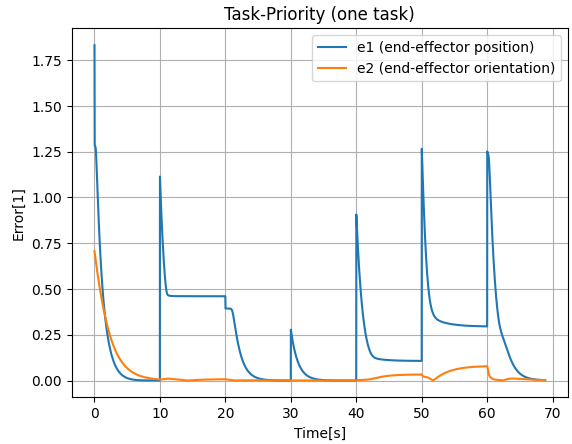
The visualization of the robot structure and its motion is shown in Fig. 4a. As shown in Fig. 4b, during certain time intervals, both tasks are satisfied and the position and orientation errors converge to zero. However, during other intervals, when the tasks are conflicting, only the end-effector position error converges to zero, while the orientation error remains high.

3.1.4 Two tasks → 1: End-effector position, 2: Joint 1 position

In this task hierarchy, the end-effector position task is assigned a higher priority than the joint 1 position task. Consequently, the robot prioritizes achieving the desired end-effector position.

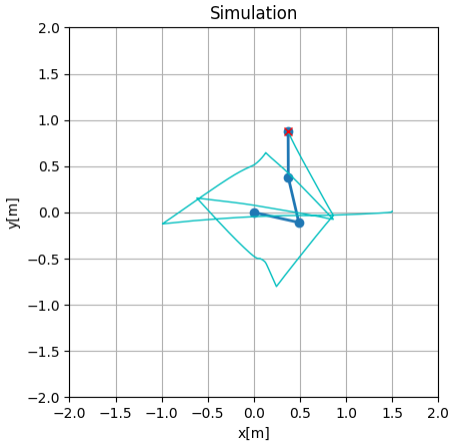


(a) Simulation of the manipulator.

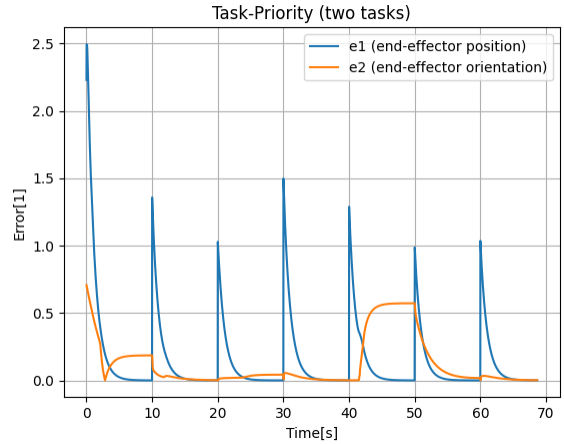


(b) Evolution of the TP control errors.

Figure 3: End-effector configuration control task.



(a) Simulation of the manipulator.

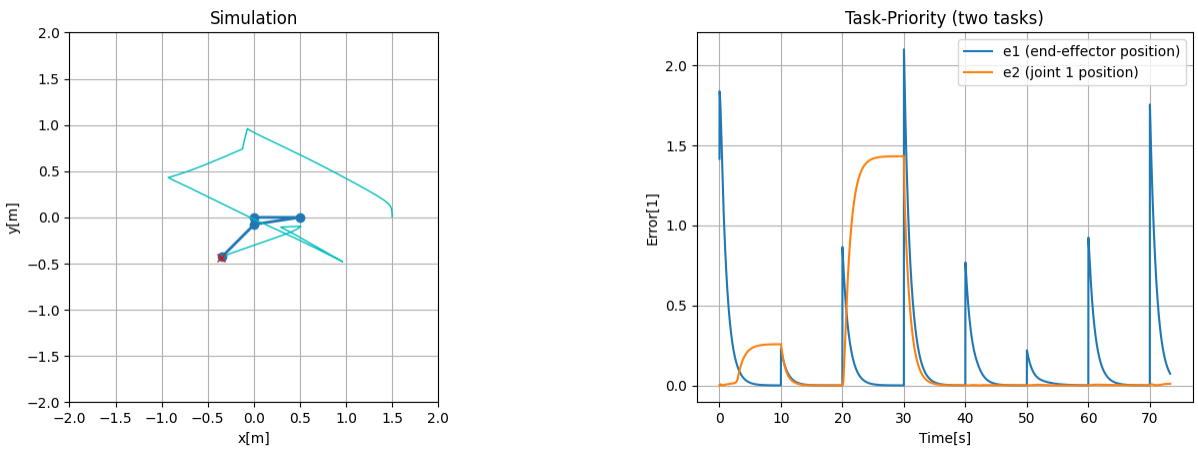


(b) Evolution of the TP control errors.

Figure 4: End-effector position and orientation control task.

When the two tasks are not in conflict, both tasks are satisfied.

The visualization of the robot structure and its motion is shown in Fig. 5a. As shown in Fig. 5b, during some time intervals, both tasks are fulfilled and the errors in end-effector position and joint 1 position converge to zero. However, when the tasks are conflicting, the end-effector position error converges to zero due to its higher priority, while the joint 1 position error remains high.



(a) Simulation of the manipulator.

(b) Evolution of the TP control errors.

Figure 5: End-effector position and joint 1 position control task.

3.2 Task-priority control with different gain matrices K

In this experiment, two tasks are defined: moving the end-effector to a desired position and maintaining joint 2 at a desired orientation. The end-effector position task is assigned a higher priority than the joint 2 orientation task. The experiment is divided into three cases, each using a different gain matrix K for the end-effector position task, in order to analyze the system behavior under varying gain values.

Before conducting the experiment, several parameters are defined and summarized in Table 3.

Desired link 2 orientation	0.0
Gain for end-effector task (case 1)	$diag[0.5, 0.5]$
Gain for end-effector task (case 2)	$diag[2.0, 2.0]$
Gain for end-effector task (case 3)	$diag[5.0, 5.0]$
Gain for link 2 orientation task	1.0
Sampling time	1/60
Total simulation time	10
Damping factor	0.1

Table 3: Parameters for the task-priority experiment

As shown in Fig. 7, since the end-effector position task has a higher priority than the link 2 orientation task, the robot prioritizes reaching the desired end-effector position whenever a conflict arises between the two tasks. This explains why, during certain time intervals, the orientation error of link 2 remains high while the end-effector position error consistently converges to zero within the 10-second simulation period.

Furthermore, it can be clearly observed that using a smaller gain matrix, $K = diag[0.5, 0.5]$, results in a significantly longer convergence time for both the end-effector position error and the link 2 orientation error when compared to $K = diag[2.0, 2.0]$ and $K = diag[5.0, 5.0]$. Among the three cases, $K = diag[5.0, 5.0]$ achieves the fastest convergence.

However, as illustrated in Fig. 6, when the gain matrix is set to $K = diag[5.0, 5.0]$, the end-effector trajectory becomes less stable compared to the other two cases. Noticeable oscillations appear in the end-effector path, indicating that although higher gain values reduce convergence time, they also increase the risk of unstable or oscillatory robot motion.

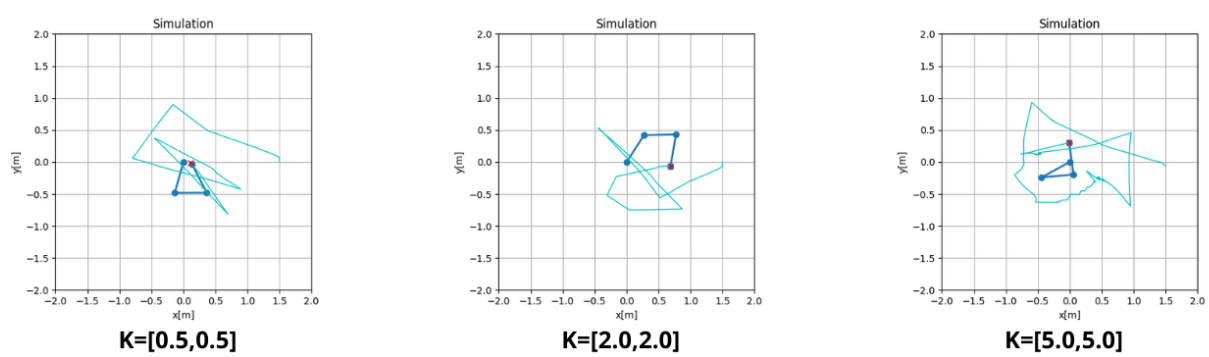


Figure 6: Simulation of the manipulator with three different gain matrix K.

The complete python implementation is presented in the Appendices A.6.1 and A.6.2.

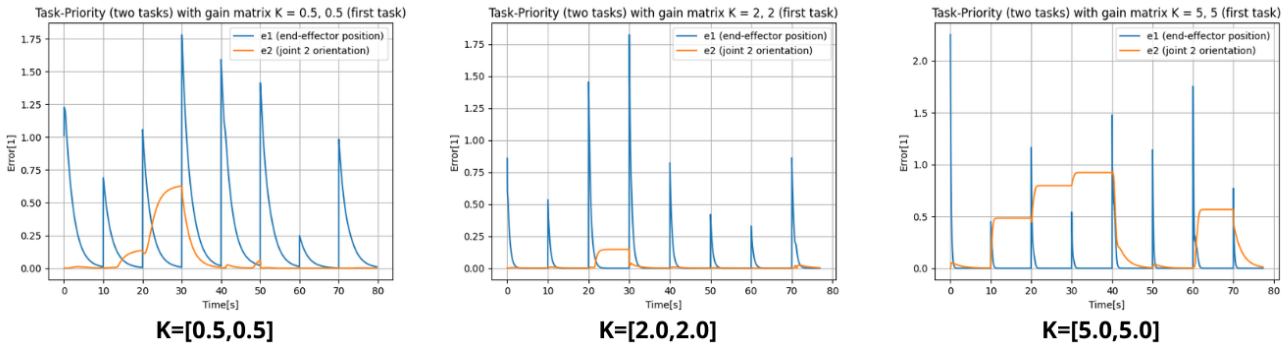


Figure 7: Evolution of the TP control errors in three different gain matrix K.

4 Conclusion

In conclusion, this laboratory exercise demonstrates the implementation of various tasks, ranging from end-effector position control to joint position control. In addition, it illustrates how feed-forward velocity combined with a gain matrix K can be used to tune the convergence time and improve tracking performance.

Through simulation and analysis of the end-effector, joint, and link position and orientation errors, the results show how the robot behavior changes depending on the defined tasks and their combinations under task-priority control. Furthermore, the influence of the gain matrix K on both the error convergence rate and the stability of the robot trajectory is clearly observed.

A Appendix: Code Implementations

This appendix contains all Python code implementations used in the laboratory exercise. To improve clarity and readability of the main report, all figures presenting source code are moved to this appendix.

A.1 End-effector position task

```python
class Position2D(Task):
    def __init__(self, name, desired):
        super().__init__(name, desired)
        self.err = np.zeros(desired.shape) # Initialize with proper dimensions

    def update(self, robot):
        self.J = robot.getEEJacobian()[0:2,:]   # Update task Jacobian
        self.err = self.getDesired() - robot.getEETransform()[0:2,
            3].reshape(2,1)# Update task error
```

A.2 End-effector orientation task

```python
class Orientation2D(Task):
    def __init__(self, name, desired):
        super().__init__(name, desired)
        self.err = 0.0 # Initialize with proper dimensions

    def update(self, robot):
        self.J = robot.getEEJacobian()[5, :].reshape(1, robot.dof) # Update
            task Jacobian
        rotation_matrix_desired = R.from_euler('z',
            self.getDesired()).as_matrix()
        w_d, epsilon_d =
            self.quaternion_from_rotation_matrix(rotation_matrix_desired)
        w, epsilon =
            self.quaternion_from_rotation_matrix(robot.getEETransform()[0:3,
            0:3])
        self.err = (w * epsilon_d - w_d * epsilon - np.cross(epsilon,
            epsilon_d))[-1]# Update task error

    def quaternion_from_rotation_matrix(self, orientation_matrix):
        orientation_matrix = R.from_matrix(orientation_matrix)
        quaternion = orientation_matrix.as_quat(scalar_first=True)
        w = quaternion[0]
        epsilon = quaternion[1:4]

        return w, epsilon
```

A.3 End-effector configuration task

```python
class Orientation2D(Task):
    def __init__(self, name, desired):
        super().__init__(name, desired)
        self.err = np.zeros(desired.shape)# Initialize with proper dimensions

    def update(self, robot):
        self.J = np.vstack((robot.getEEJacobian()[0:2,:],
            robot.getEEJacobian()[5, :]))# Update task Jacobian
        rotation_matrix_desired = R.from_euler('z',
            self.getDesired()[-1]).as_matrix()
        w_d, epsilon_d =
            self.quaternion_from_rotation_matrix(rotation_matrix_desired)
        w, epsilon =
            self.quaternion_from_rotation_matrix(robot.getEETransform()[0:3,
            0:3])
        self.err = np.vstack((self.getDesired()[0:2].reshape(2,1) -
            robot.getEETransform()[0:2, 3].reshape(2,1),
                                (w * epsilon_d - w_d * epsilon -
                                    np.cross(epsilon, epsilon_d))[-1]))# Update
                                    task error

    def quaternion_from_rotation_matrix(self, orientation_matrix):
        orientation_matrix = R.from_matrix(orientation_matrix)
        quaternion = orientation_matrix.as_quat(scalar_first=True)
        w = quaternion[0]
        epsilon = quaternion[1:4]

        return w, epsilon
```

A.4 Joint position task

```python
class JointPosition(Task):
    def __init__(self, name, index, desired):
        super().__init__(name, desired)
        self.err = 0.0
        self.index = index


    def update(self, robot):
        self.J = np.zeros((1, robot.getDOF())) # Update task Jacobian
        self.J[0, self.index] = 1
        self.err = self.getDesired() - robot.getJointPos(self.index)
```

A.5 Full code for four different hierarchies task

A.5.1 lab4_robotics.py

```
from lab2_robotics import * # Includes numpy import
from scipy.spatial.transform import Rotation as R

def jacobianLink(T, revolute, link): # Needed in Exercise 2
    '''
        Function builds a Jacobian for the end-effector of a robot,
        described by a list of kinematic transformations and a list of joint
            types.

        Arguments:
        T (list of Numpy array): list of transformations along the kinematic
            chain of the robot (from the base frame)
        revolute (list of Bool): list of flags specifying if the corresponding
            joint is a revolute joint
        link(integer): index of the link for which the Jacobian is computed

        Returns:
        (Numpy array): end-effector Jacobian
    '''
    # 1. Initialize J and O.
    n_dof = len(revolute)
    J = np.zeros((6, n_dof - link + 1))
    O = T[-1][0:3, 3]
    # 2. For each joint of the robot
    for i in range(link-1, n_dof):
        #   a. Extract z and o.
        i_1_T_i = T[i]
        z = i_1_T_i[0:3, 2]
        o = i_1_T_i[0:3, 3]
        #   b. Check joint type.
        if revolute[i] == True:
            cross_vector = np.array([np.cross(z, (O - o))]).T
            z = np.array([z]).T
            J_i = np.vstack((cross_vector, z))
        else:
            J_i = np.vstack((z, np.zeros((3, 1))))
        #   c. Modify corresponding column of J.
        J[:, i] = J_i[:, 0]
    return J


'''
    Class representing a robotic manipulator.
'''
class Manipulator:
    '''
        Constructor.
```

```python

        Arguments:
        d (Numpy array): list of displacements along Z-axis
        theta (Numpy array): list of rotations around Z-axis
        a (Numpy array): list of displacements along X-axis
        alpha (Numpy array): list of rotations around X-axis
        revolute (list of Bool): list of flags specifying if the corresponding joint is a revolute joint
    '''
    def __init__(self, d, theta, a, alpha, revolute):
        self.d = d
        self.theta = theta
        self.a = a
        self.alpha = alpha
        self.revolute = revolute
        self.dof = len(self.revolute)
        self.q = np.zeros(self.dof).reshape(-1, 1)
        self.update(0.0, 0.0)

    '''
        Method that updates the state of the robot.

        Arguments:
        dq (Numpy array): a column vector of joint velocities
        dt (double): sampling time
    '''
    def update(self, dq, dt):
        self.q += dq * dt
        for i in range(len(self.revolute)):
            if self.revolute[i]:
                self.theta[i] = self.q[i][0]
            else:
                self.d[i] = self.q[i][0]
        self.T = kinematics(self.d, self.theta, self.a, self.alpha)

    '''
        Method that returns the characteristic points of the robot.
    '''
    def drawing(self):
        return robotPoints2D(self.T)

    '''
        Method that returns the end-effector Jacobian.
    '''
    def getEEJacobian(self):
        return jacobian(self.T, self.revolute)

    '''
        Method that returns the end-effector transformation.
    '''
    def getEETransform(self):
```

```
        return self.T[-1]

    '''
        Method that returns the position of a selected joint.

        Argument:
        joint (integer): index of the joint

        Returns:
        (double): position of the joint
    '''
    def getJointPos(self, joint):
        return self.q[joint]

    '''
        Method that returns number of DOF of the manipulator.
    '''
    def getDOF(self):
        return self.dof

'''
    Base class representing an abstract Task.
'''
class Task:
    '''
        Constructor.

        Arguments:
        name (string): title of the task
        desired (Numpy array): desired sigma (goal)
    '''
    def __init__(self, name, desired):
        self.name = name # task title
        self.sigma_d = desired # desired sigma

    '''
        Method updating the task variables (abstract).

        Arguments:
        robot (object of class Manipulator): reference to the manipulator
    '''
    def update(self, robot):
        pass

    '''
        Method setting the desired sigma.

        Arguments:
        value(Numpy array): value of the desired sigma (goal)
    '''
    def setDesired(self, value):
```

```
        self.sigma_d = value

    '''
        Method returning the desired sigma.
    '''
    def getDesired(self):
        return self.sigma_d

    '''
        Method returning the task Jacobian.
    '''
    def getJacobian(self):
        return self.J

    '''
        Method returning the task error (tilde sigma).
    '''
    def getError(self):
        return self.err

'''
    Subclass of Task, representing the 2D position task.
'''
class Position2D(Task):
    def __init__(self, name, desired):
        super().__init__(name, desired)
        self.err = np.zeros(desired.shape) # Initialize with proper dimensions

    def update(self, robot):
        self.J = robot.getEEJacobian()[0:2,:]    # Update task Jacobian
        self.err = self.getDesired() - robot.getEETransform()[0:2,
            3].reshape(2,1)# Update task error

'''
    Subclass of Task, representing the 2D orientation task.
'''
class Orientation2D(Task):
    def __init__(self, name, desired):
        super().__init__(name, desired)
        self.err = 0.0 # Initialize with proper dimensions

    def update(self, robot):
        self.J = robot.getEEJacobian()[5, :].reshape(1, robot.dof) # Update
            task Jacobian
        rotation_matrix_desired = R.from_euler('z',
            self.getDesired()).as_matrix()
        w_d, epsilon_d =
            self.quaternion_from_rotation_matrix(rotation_matrix_desired)
        w, epsilon =
            self.quaternion_from_rotation_matrix(robot.getEETransform()[0:3,
            0:3])
```

```
        self.err = (w * epsilon_d - w_d * epsilon - np.cross(epsilon, epsilon_d))[-1]# Update task error

    def quaternion_from_rotation_matrix(self, orientation_matrix):
        orientation_matrix = R.from_matrix(orientation_matrix)
        quaternion = orientation_matrix.as_quat(scalar_first=True)
        w = quaternion[0]
        epsilon = quaternion[1:4]

        return w, epsilon
'''
    Subclass of Task, representing the 2D configuration task.
'''
class Configuration2D(Task):
    def __init__(self, name, desired):
        super().__init__(name, desired)
        self.err = np.zeros(desired.shape)# Initialize with proper dimensions

    def update(self, robot):
        self.J = np.vstack((robot.getEEJacobian()[0:2,:], robot.getEEJacobian()[5, :]))# Update task Jacobian
        rotation_matrix_desired = R.from_euler('z', self.getDesired()[-1]).as_matrix()
        w_d, epsilon_d = self.quaternion_from_rotation_matrix(rotation_matrix_desired)
        w, epsilon = self.quaternion_from_rotation_matrix(robot.getEETransform()[0:3, 0:3])
        self.err = np.vstack((self.getDesired()[0:2].reshape(2,1) - robot.getEETransform()[0:2, 3].reshape(2,1),
                              (w * epsilon_d - w_d * epsilon - np.cross(epsilon, epsilon_d))[-1]))# Update task error

    def quaternion_from_rotation_matrix(self, orientation_matrix):
        orientation_matrix = R.from_matrix(orientation_matrix)
        quaternion = orientation_matrix.as_quat(scalar_first=True)
        w = quaternion[0]
        epsilon = quaternion[1:4]

        return w, epsilon
'''
    Subclass of Task, representing the joint position task.
'''
class JointPosition(Task):
    def __init__(self, name, index, desired):
        super().__init__(name, desired)
        self.err = 0.0
        self.index = index


```

```
    def update(self, robot):
        self.J = np.zeros((1, robot.getDOF())) # Update task Jacobian
        self.J[0, self.index] = 1
        self.err = self.getDesired() - robot.getJointPos(self.index)
```

A.5.2 lab4_recursive_tp.py

```
from lab4_robotics import * # Includes numpy import
import matplotlib.pyplot as plt
import matplotlib.animation as anim

# Robot model - 3-link manipulator
d = np.zeros(3)                             # displacement along Z-axis
theta = np.array([0.2, 0.5, 0.2])           # rotation around Z-axis
alpha = np.zeros(3)                         # rotation around X-axis
a = np.array([0.5, 0.5, 0.5])               # displacement along X-axis
revolute = np.array([True, True, True])     # flags specifying the type of joints
robot = Manipulator(d, theta, a, alpha, revolute) # Manipulator object
damping_factor = 0.1

# Task hierarchy definition
tasks = [
            Position2D("End-effector position", np.array([1.0, 0.5]).reshape(2,1)),
            # Orientation2D("End-effector orientation", np.pi/2),
            # Configuration2D("Configuration", np.array([1.0, 0.5, np.pi/2]).reshape(3,1)),
            JointPosition("Joint position", 0, 0)
        ]

# Simulation params
dt = 1.0/60.0

# Drawing preparation
fig = plt.figure()
ax = fig.add_subplot(111, autoscale_on=False, xlim=(-2, 2), ylim=(-2,2))
ax.set_title('Simulation')
ax.set_aspect('equal')
ax.grid()
ax.set_xlabel('x[m]')
ax.set_ylabel('y[m]')
line, = ax.plot([], [], 'o-', lw=2) # Robot structure
path, = ax.plot([], [], 'c-', lw=1) # End-effector path
point, = ax.plot([], [], 'rx') # Target
PPx = []
PPy = []

# Record data for plotting
error_end_effector = []
```

```python
error_end_effector_orientation = []
error_joint_position = []

# Simulation initialization
def init():
    global tasks
    line.set_data([], [])
    path.set_data([], [])
    point.set_data([], [])
    if len(tasks[0].getDesired()) == 2:
        tasks[0].setDesired((np.random.rand(2,1)*2-1)) # Random target
    else:
        angle = np.array([tasks[0].getDesired()[-1]])
        sigma_d = np.hstack((np.random.rand(1,2)*2-1, angle))
        tasks[0].setDesired(sigma_d.reshape(3,1)) # Random target
    return line, path, point

# Simulation loop
def simulate(t):
    global tasks
    global robot
    global PPx, PPy

    ### Recursive Task-Priority algorithm
    # Initialize null-space
    P = np.identity(robot.dof)
    # Initialize output vector (joint velocity)
    dq = np.zeros((robot.dof, 1))
    # Loop over tasks
    for i in range(len(tasks)):
        # Update task state
        tasks[i].update(robot)
        # Compute augmented Jacobian
        J_bar = tasks[i].getJacobian() @ P
        # Compute task velocity & Accumulate velocity
        dq = dq + DLS(J_bar, damping_factor) @ (tasks[i].getError() -
            tasks[i].getJacobian() @ dq)
        # Update null-space projector
        P = P - np.linalg.pinv(J_bar) @ J_bar
    ###

    # Record data for plotting
    if tasks[0].name == "Configuration":
        error_end_effector.append(np.linalg.norm(tasks[0].getError()[0:2]))
        error_end_effector_orientation.append(abs(tasks[0].getError()[2]))
    elif len(tasks) == 1 and tasks[0].name == "End-effector position":
        error_end_effector.append(np.linalg.norm(tasks[0].getError()[0:2]))
    elif len(tasks) > 1 and tasks[1].name == "Joint position":
        error_end_effector.append(np.linalg.norm(tasks[0].getError()[0:2]))
        error_joint_position.append(abs(tasks[1].getError()))
    elif len(tasks) > 1 and tasks[1].name == "End-effector orientation":
```

```
        error_end_effector.append(np.linalg.norm(tasks[0].getError()[0:2]))
        error_end_effector_orientation.append(abs(tasks[1].getError()))


    # Update robot
    robot.update(dq, dt)

    # Update drawing
    PP = robot.drawing()
    line.set_data(PP[0,:], PP[1,:])
    PPx.append(PP[0,-1])
    PPy.append(PP[1,-1])
    path.set_data(PPx, PPy)
    point.set_data(tasks[0].getDesired()[0], tasks[0].getDesired()[1])

    return line, path, point

# Run simulation
animation = anim.FuncAnimation(fig, simulate, np.arange(0, 10, dt),
                                interval=10, blit=True, init_func=init,
                                repeat=True)
plt.show()

# Plot errors
tvec = np.arange(0, len(error_end_effector)*dt, dt)
plt.figure()
if tasks[0].name == "Configuration":
    plt.plot(tvec, error_end_effector, label='End-effector position error')
    plt.plot(tvec, error_end_effector_orientation, label='End-effector orientation error')
elif len(tasks) == 1 and tasks[0].name == "End-effector position":
    plt.plot(tvec, error_end_effector, label='End-effector position error')
elif len(tasks) > 1 and tasks[1].name == "Joint position":
    plt.plot(tvec, error_end_effector, label='End-effector position error')
    plt.plot(tvec, error_joint_position, label='Joint position error')
elif len(tasks) > 1 and tasks[1].name == "End-effector orientation":
    plt.plot(tvec, error_end_effector, label='End-effector position error')
    plt.plot(tvec, error_end_effector_orientation, label='End-effector orientation error')
plt.xlabel('Time step')
plt.ylabel('Error')
plt.title('Task Errors')
plt.legend()
plt.grid()
plt.show()
```

A.6 Full code for task-priority with different gain matrix K

A.6.1 lab4_robotics.py

```
from lab2_robotics import * # Includes numpy import
from scipy.spatial.transform import Rotation as R

def jacobianLink(T, revolute, link): # Needed in Exercise 2
    '''
        Function builds a Jacobian for the end-effector of a robot,
        described by a list of kinematic transformations and a list of joint
            types.

        Arguments:
        T (list of Numpy array): list of transformations along the kinematic
            chain of the robot (from the base frame)
        revolute (list of Bool): list of flags specifying if the corresponding
            joint is a revolute joint
        link(integer): index of the link for which the Jacobian is computed

        Returns:
        (Numpy array): end-effector Jacobian
    '''
    # 1. Initialize J and O.
    J = np.zeros((6, link))
    O = T[-1][0:3, 3]
    # 2. For each joint of the robot
    for i in range(0, link):
        #   a. Extract z and o.
        i_1_T_i = T[i]
        z = i_1_T_i[0:3, 2]
        o = i_1_T_i[0:3, 3]
        #   b. Check joint type.
        if revolute[i] == True:
            cross_vector = np.array([np.cross(z, (O - o))]).T
            z = np.array([z]).T
            J_i = np.vstack((cross_vector, z))
        else:
            J_i = np.vstack((z, np.zeros((3, 1))))
        #   c. Modify corresponding column of J.
        J[:, i] = J_i[:, 0]
    return J


'''
    Class representing a robotic manipulator.
'''
class Manipulator:
    '''
        Constructor.

```

```
        Arguments:
        d (Numpy array): list of displacements along Z-axis
        theta (Numpy array): list of rotations around Z-axis
        a (Numpy array): list of displacements along X-axis
        alpha (Numpy array): list of rotations around X-axis
        revolute (list of Bool): list of flags specifying if the corresponding
            joint is a revolute joint
    '''
    def __init__(self, d, theta, a, alpha, revolute):
        self.d = d
        self.theta = theta
        self.a = a
        self.alpha = alpha
        self.revolute = revolute
        self.dof = len(self.revolute)
        self.q = np.zeros(self.dof).reshape(-1, 1)
        self.update(0.0, 0.0)

    '''
        Method that updates the state of the robot.

        Arguments:
        dq (Numpy array): a column vector of joint velocities
        dt (double): sampling time
    '''
    def update(self, dq, dt):
        self.q += dq * dt
        for i in range(len(self.revolute)):
            if self.revolute[i]:
                self.theta[i] = self.q[i][0]
            else:
                self.d[i] = self.q[i][0]
        self.T = kinematics(self.d, self.theta, self.a, self.alpha)

    '''
        Method that returns the characteristic points of the robot.
    '''
    def drawing(self):
        return robotPoints2D(self.T)

    '''
        Method that returns the end-effector Jacobian.
    '''
    def getEEJacobian(self):
        return jacobian(self.T, self.revolute)

    '''
        Method that returns the end-effector transformation.
    '''
    def getEETransform(self):
        return self.T[-1]
```

```python

    '''
        Method that returns the transformation of a selected link.
    '''
    def getLinkTransform(self, link):
        return self.T[link]

    '''
        Method that returns the Jacobian of a selected link.
    '''
    def getLinkJacobian(self, link):
        return jacobianLink(self.T, self.revolute, link)

    '''
        Method that returns the position of a selected joint.

        Argument:
        joint (integer): index of the joint

        Returns:
        (double): position of the joint
    '''
    def getJointPos(self, joint):
        return self.q[joint]

    '''
        Method that returns number of DOF of the manipulator.
    '''
    def getDOF(self):
        return self.dof

'''
    Base class representing an abstract Task.
'''
class Task:
    '''
        Constructor.

        Arguments:
        name (string): title of the task
        desired (Numpy array): desired sigma (goal)
    '''
    def __init__(self, name, desired):
        self.name = name # task title
        self.sigma_d = desired # desired sigma
        self.ff_vel = None
        self.gain_matrix_K = None

    '''
        Method updating the task variables (abstract).

```

```
        Arguments:
        robot (object of class Manipulator): reference to the manipulator
    '''
    def update(self, robot):
        pass

    '''
        Method setting the desired sigma.

        Arguments:
        value(Numpy array): value of the desired sigma (goal)
    '''
    def setDesired(self, value):
        self.sigma_d = value

    '''
        Method returning the desired sigma.
    '''
    def getDesired(self):
        return self.sigma_d

    '''
        Method setting the feedforward velocity.

        Arguments:
        value(Numpy array): value of the feedforward velocity vector
    '''
    def setFeedforwardVelocity(self, value):
        self.ff_vel = value

    '''
        Method returning the feedforward velocity vector.
    '''
    def getFeedforwardVelocity(self):
        return self.ff_vel

    '''
        Method setting the gain matrix K.

        Arguments:
        value(Numpy array): value of the gain matrix K
    '''
    def setGainMatrixK(self, value):
        self.gain_matrix_K = value

    '''
        Method returning the gain matrix K.
    '''
    def getGainMatrixK(self):
        return self.gain_matrix_K

```

```
    '''
        Method returning the task Jacobian.
    '''
    def getJacobian(self):
        return self.J

    '''
        Method returning the task error (tilde sigma).
    '''
    def getError(self):
        return self.err

'''
    Subclass of Task, representing the 2D position task.
'''
class Position2D(Task):
    def __init__(self, name, desired, link_index):
        super().__init__(name, desired)
        self.err = np.zeros(desired.shape) # Initialize with proper dimensions
        self.link_index = link_index # Index of the link for which the task is defined
        self.ff_vel = np.zeros(desired.shape) # Initialize feedforward velocity
        self.gain_matrix_K = np.eye(desired.shape[0]) # Initialize gain matrix K

    def update(self, robot):
        self.J = robot.getLinkJacobian(self.link_index)[0:2,:]   # Update task Jacobian
        self.err = self.getDesired() - robot.getLinkTransform(self.link_index)[0:2, 3].reshape(2,1)# Update task error

'''
    Subclass of Task, representing the 2D orientation task.
'''
class Orientation2D(Task):
    def __init__(self, name, desired, link_index):
        super().__init__(name, desired)
        self.err = 0.0 # Initialize with proper dimensions
        self.link_index = link_index # Index of the link for which the task is defined
        self.ff_vel = np.zeros(1) # Initialize feedforward velocity
        self.gain_matrix_K = np.eye(1) # Initialize gain matrix K

    def update(self, robot):
        self.J = robot.getLinkJacobian(self.link_index)[5, :].reshape(1, self.link_index) # Update task Jacobian
        rotation_matrix_desired = R.from_euler('z', self.getDesired()).as_matrix() # Convert desired orientation from Euler angles to rotation matrix
```

```
        w_d, epsilon_d = self.quaternion_from_rotation_matrix(rotation_matrix_desired)
        w, epsilon = self.quaternion_from_rotation_matrix(robot.getLinkTransform(self.link_index)[0:3])
        self.err = (w * epsilon_d - w_d * epsilon - np.cross(epsilon, epsilon_d))[-1]# Update task error
        self.err = np.array(self.err).reshape(1, 1) # Reshape error to be a column vector

    # Method to convert rotation matrix to quaternion (scalar-first convention)
    def quaternion_from_rotation_matrix(self, orientation_matrix):
        orientation_matrix = R.from_matrix(orientation_matrix)
        quaternion = orientation_matrix.as_quat(scalar_first=True)
        w = quaternion[0]
        epsilon = quaternion[1:4]

        return w, epsilon
'''
    Subclass of Task, representing the 2D configuration task.
'''
class Configuration2D(Task):
    def __init__(self, name, desired, link_index):
        super().__init__(name, desired)
        self.err = np.zeros(desired.shape) # Initialize with proper dimensions
        self.link_index = link_index # Index of the link for which the task is defined
        self.ff_vel = np.zeros(desired.shape) # Initialize feedforward velocity
        self.gain_matrix_K = np.eye(desired.shape[0]) # Initialize gain matrix K

    def update(self, robot):
        self.J = np.vstack((robot.getLinkJacobian(self.link_index)[0:2,:], robot.getLinkJacobian(self.link_index)[5, :])).reshape(3, self.link_index)# Update task Jacobian
        rotation_matrix_desired = R.from_euler('z', self.getDesired()[-1]).as_matrix() # Convert desired orientation from Euler angles to rotation matrix
        w_d, epsilon_d = self.quaternion_from_rotation_matrix(rotation_matrix_desired) # Desired orientation in quaternion form
        w, epsilon = self.quaternion_from_rotation_matrix(robot.getLinkTransform(self.link_index)[0:3]) # Current orientation in quaternion form
        self.err = np.vstack((self.getDesired()[0:2].reshape(2,1) - robot.getLinkTransform(self.link_index)[0:2, 3].reshape(2,1),
                              (w * epsilon_d - w_d * epsilon - np.cross(epsilon, epsilon_d))[-1]))# Update task error
        self.err = self.err.reshape(3, 1) # Reshape error to be a column vector

```

```
    def quaternion_from_rotation_matrix(self, orientation_matrix):
        orientation_matrix = R.from_matrix(orientation_matrix)
        quaternion = orientation_matrix.as_quat(scalar_first=True)
        w = quaternion[0]
        epsilon = quaternion[1:4]

        return w, epsilon
'''
    Subclass of Task, representing the joint position task.
'''
class JointPosition(Task):
    def __init__(self, name, index, desired):
        super().__init__(name, desired)
        self.err = 0.0 # Initialize with proper dimensions
        self.index = index # Index of the joint for which the task is defined
        self.ff_vel = np.zeros(1) # Initialize feedforward velocity
        self.gain_matrix_K = np.eye(1) # Initialize gain matrix K


    def update(self, robot):
        self.J = np.zeros((1, robot.getDOF())) # Update task Jacobian
        self.J[0, self.index] = 1 # Set the column corresponding to the joint index to 1
        self.err = self.getDesired() - robot.getJointPos(self.index) # Update task error
        self.err = np.array(self.err).reshape(1, 1) # Reshape error to be a column vector
```

A.6.2 lab4_recursive_tp.py

```
from lab4_robotics import * # Includes numpy import
import matplotlib.pyplot as plt
import matplotlib.animation as anim

# Robot model - 3-link manipulator
d = np.zeros(3)                             # displacement along Z-axis
theta = np.array([0.2, 0.5, 0.2])           # rotation around Z-axis
alpha = np.zeros(3)                         # rotation around X-axis
a = np.array([0.5, 0.5, 0.5])               # displacement along X-axis
revolute = np.array([True, True, True])     # flags specifying the type of joints
robot = Manipulator(d, theta, a, alpha, revolute) # Manipulator object
damping_factor = 0.1

# Three gain matrices for first task
Kp1 = np.diag([0.5, 0.5]) # Gain matrix for end-effector position control
Kp2 = np.diag([2, 2]) # Gain matrix for end-effector position control
Kp3 = np.diag([5, 5]) # Gain matrix for end-effector position control
Kp = Kp3 # We can test with Kp1, Kp2, or Kp3 to see the effect of gain matrix on the convergence speed
```

```python

# Task hierarchy definition
tasks = [
            Position2D("End-effector position", np.array([1.0, 0.5]).reshape(2,1), link_index = 3),
            Orientation2D("End-effector orientation", 0, link_index = 2),
            # Configuration2D("Configuration", np.array([1.0, 0.5, np.pi/2]).reshape(3,1), 3),
            # JointPosition("Joint position", 0, 0)
        ]

# Simulation params
dt = 1.0/60.0

# Drawing preparation
fig = plt.figure()
ax = fig.add_subplot(111, autoscale_on=False, xlim=(-2, 2), ylim=(-2,2))
ax.set_title('Simulation')
ax.set_aspect('equal')
ax.grid()
ax.set_xlabel('x[m]')
ax.set_ylabel('y[m]')
line, = ax.plot([], [], 'o-', lw=2) # Robot structure
path, = ax.plot([], [], 'c-', lw=1) # End-effector path
point, = ax.plot([], [], 'rx') # Target
PPx = []
PPy = []

# Record data for plotting
error_end_effector = []
error_end_effector_orientation = []
error_joint_position = []

# Simulation initialization
def init():
    global tasks
    line.set_data([], [])
    path.set_data([], [])
    point.set_data([], [])
    if len(tasks[0].getDesired()) == 2:
        tasks[0].setDesired((np.random.rand(2,1)*2-1)) # Random target
        tasks[0].setGainMatrixK(Kp) # Set gain matrix for the first task
    else:
        angle = np.array([tasks[0].getDesired()[-1]])
        sigma_d = np.hstack((np.random.rand(1,2)*2-1, angle))
        tasks[0].setDesired(sigma_d.reshape(3,1)) # Random target
    return line, path, point

# Simulation loop
def simulate(t):
    global tasks
```

```
    global robot
    global PPx, PPy

    ### Recursive Task-Priority algorithm
    # Initialize null-space
    P = np.identity(robot.dof)
    # Initialize output vector (joint velocity)
    dq = np.zeros((robot.dof, 1))
    # Loop over tasks
    for i in range(len(tasks)):
        # Update task state
        tasks[i].update(robot)
        # Padding Jacobian to match the dimension of the robot dof and compute the augmented Jacobian, then compute the DLS solution and accumulate velocity, and finally update the null-space projector
        Jacobian_padded = np.pad(tasks[i].getJacobian(), ((0, 0), (0, robot.dof - tasks[i].getJacobian().shape[1])))
        # Compute augmented Jacobian
        J_bar = Jacobian_padded @ P
        # Task definition
        x_dot = tasks[i].getFeedforwardVelocity() + tasks[i].getGainMatrixK() @ tasks[i].getError()
        # Compute task velocity & Accumulate velocity
        dq = dq + DLS(J_bar, damping_factor) @ (x_dot - Jacobian_padded @ dq)
        # Update null-space projector
        P = P - np.linalg.pinv(J_bar) @ J_bar
    ###

    # Record data for plotting
    if tasks[0].name == "Configuration":
        error_end_effector.append(np.linalg.norm(tasks[0].getError()[0:2]))
        error_end_effector_orientation.append(abs(tasks[0].getError()[2]))
    elif len(tasks) == 1 and tasks[0].name == "End-effector position":
        error_end_effector.append(np.linalg.norm(tasks[0].getError()[0:2]))
    elif len(tasks) > 1 and tasks[1].name == "Joint position":
        error_end_effector.append(np.linalg.norm(tasks[0].getError()[0:2]))
        error_joint_position.append(abs(tasks[1].getError()[0]))
    elif len(tasks) > 1 and tasks[1].name == "End-effector orientation":
        error_end_effector.append(np.linalg.norm(tasks[0].getError()[0:2]))
        error_end_effector_orientation.append(abs(tasks[1].getError()[0]))


    # Update robot
    robot.update(dq, dt)

    # Update drawing
    PP = robot.drawing()
    line.set_data(PP[0,:], PP[1,:])
    PPx.append(PP[0,-1])
    PPy.append(PP[1,-1])
    path.set_data(PPx, PPy)
```

```
    point.set_data(tasks[0].getDesired()[0], tasks[0].getDesired()[1])

    return line, path, point

# Run simulation
animation = anim.FuncAnimation(fig, simulate, np.arange(0, 10, dt),
                                interval=10, blit=True, init_func=init,
                                repeat=True)
plt.show()

# Plot errors
tvec = np.arange(0, len(error_end_effector)*dt, dt)
plt.figure()
if tasks[0].name == "Configuration":
    plt.plot(tvec, error_end_effector, label='e1 (end-effector position)')
    plt.plot(tvec, error_end_effector_orientation, label='e2 (end-effector orientation)')
elif len(tasks) == 1 and tasks[0].name == "End-effector position":
    plt.plot(tvec, error_end_effector, label='e1 (end-effector position)')
elif len(tasks) > 1 and tasks[1].name == "Joint position":
    plt.plot(tvec, error_end_effector, label='e1 (end-effector position)')
    plt.plot(tvec, error_joint_position, label='e2 (joint 1 position)')
elif len(tasks) > 1 and tasks[1].name == "End-effector orientation":
    plt.plot(tvec, error_end_effector, label='e1 (end-effector position)')
    plt.plot(tvec, error_end_effector_orientation, label='e2 (joint 2 orientation)')
plt.xlabel('Time[s]')
plt.ylabel('Error[1]')
plt.title('Task-Priority (two tasks) with gain matrix K = ' + str(Kp[0,0]) + ', ' + str(Kp[1,1]) + ' (first task)')
plt.legend()
plt.grid()
plt.show()
```